BitDB — A Bitcask-Style Key-Value Store

Parallel Computing Laboratory (20MSSL02)

Shree Kottes J (7176 22 31 050)

April 2026

Problem Statement

The Cold-Start Problem in Key-Value Stores

- Disk-based KV stores lose their in-memory index on every restart
- On startup, the **entire dataset must be scanned** to rebuild the index
- With millions of records across hundreds of files, this is **sequential I/O**
- A 1 M-key database at default settings requires decoding every record — **one file at a time**
  Cold-start latency grows linearly with dataset size — can we do better?

Existing Approach

Bitcask — The Original Design

Bitcask (Riak, 2010) introduced the *append-only + in-memory hash index* model:

Concept	How it works
Data files	Append-only segments; never modified in place
KeyDir	In-memory `HashMap<key -> (file_id, offset, size)>`
Reads	O(1) — one hash lookup + one disk seek
Writes	O(1) — append to active file
Startup	Scan **all** data files top-to-bottom, **sequentially**
Hint files	Optional index sidecars to skip full record decoding

The bottleneck: startup rebuild is strictly serial — file N cannot start until file N-1 finishes.

What We Are Solving

Serial Startup Rebuild is Unnecessarily Slow

Each data file is **independent** — records in `data.0003` have no dependency on `data.0002`.

Yet the classic implementation processes them one at a time:

```
Thread 0: [f0]->[f1]->[f2]->[f3]->  done
```

Observation: File scanning is embarrassingly parallel.

- No shared mutable state between files during the read phase
- The only ordering constraint is in the *merge step* — last writer wins per key
- Same applies to **compaction (merge)** — reading live values from disk

Our Solution

Parallel KeyDir Rebuild with Rayon

Split the work across all available CPU cores:

```
Thread 0: [f0]-[f4]-[f8]  -> entries[]
Thread 1: [f1]-[f5]-[f9]  -> entries[]
Thread 2: [f2]-[f6]-[f10] -> entries[]
Thread 3: [f3]-[f7]-[f11] -> entries[]
                  |
           sort by (file_id, offset)
                  |
           single-pass KeyDir build
```

Correctness preserved: entries are sorted by `(file_id ASC, offset ASC)` before insertion — identical outcome to the serial path; *last writer always wins*.

Implementation

Startup Rebuild (recovery.rs)

```
// Each file scanned independently on a rayon worker
file_info.par_iter().map(|f| scan_file(f)).collect()

// Sort by (file_id, offset) → deterministic KeyDir build
all_entries.sort_unstable_by_key(
    |(fid, off, ..)| (*fid, *off)
);
```

Implementation (cont.)

Merge / Compaction (`merge.rs`)

```
// Read phase: parallel value loading from disk
live_entries.par_iter().map(|e| load_value(e)).collect()
// Write phase: always serial (single FileSet writer)
```

Parallelism Config (`Options`)

Value	Behaviour
`Serial`	Single-threaded, deterministic
`Auto`	Rayon uses all logical CPUs
`Fixed(n)`	Exactly *n* threads

The Tracer

Visualising the Parallel Rebuild — Live

`tracer` is a companion TUI that makes the parallelism **visible**:

- Generates a configurable dataset (keys, value size, file size)
- Runs **serial rebuild** then **parallel rebuild** on the same dataset
- Shows each thread's file assignments as live progress bars
- Displays wall-clock timing, per-file duration, and speedup in real time

```
+-- SERIAL (1 thread) - done -----------------+
| v Thread 0                                  |
|   f:000 [#####################] 5,714 keys  |
|   f:001 [#####################] 5,714 keys  |
|   ...                                       |
+---------------------------------------------+
```

Results — Tracer Screenshot

Live Side-by-Side View

```
bitdb tracer   keys:5,000,000   threads:7
SERIAL (1 thread) - done
▾ Thread 0
  f:001 [                    ]  2363us  1,783 keys
  f:002 [                    ]  2103us  1,783 keys
  f:003 [                    ]  1918us  1,783 keys
  f:004 [                    ]  1916us  1,783 keys
  f:005 [                    ]  1919us  1,783 keys
  f:006 [                    ]  1925us  1,783 keys
  f:007 [                    ]  1920us  1,783 keys
  f:008 [                    ]  1936us  1,783 keys
  f:009 [                    ]  2075us  1,783 keys
  f:010 [                    ]  1937us  1,783 keys
  f:011 [                    ]  1936us  1,783 keys
  f:012 [                    ]  1932us  1,783 keys
  f:013 [                    ]  1948us  1,783 keys
  f:014 [                    ]  1938us  1,783 keys
  f:015 [                    ]  1939us  1,783 keys
  f:016 [                    ]  1944us  1,783 keys
  f:017 [                    ]  1955us  1,783 keys
  f:018 [                    ]  1967us  1,783 keys
  f:019 [                    ]  1938us  1,783 keys
  f:020 [                    ]  1940us  1,783 keys
  f:021 [                    ]  1960us  1,783 keys
  f:022 [                    ]  2034us  1,783 keys
  f:023 [                    ]  1948us  1,783 keys
  f:024 [                    ]  1948us  1,783 keys
  f:025 [                    ]  1946us  1,783 keys
  f:026 [                    ]  1942us  1,783 keys
  f:027 [                    ]  1937us  1,783 keys
  f:028 [                    ]  1950us  1,783 keys
  f:029 [                    ]  1944us  1,783 keys
  f:030 [                    ]  1952us  1,783 keys
  f:031 [                    ]  1945us  1,783 keys
  f:032 [                    ]  1963us  1,783 keys
  f:033 [                    ]  1946us  1,783 keys
  f:034 [                    ]  1948us  1,783 keys
  f:035 [                    ]  1940us  1,783 keys
  f:036 [                    ]  1959us  1,783 keys
  f:037 [                    ]  1948us  1,783 keys

PARALLEL (7 threads) - done
▾ Thread 0
  f:001 [                    ]  3067us  1,783 keys
  f:008 [                    ]  2278us  1,783 keys
  f:015 [                    ]  2125us  1,783 keys
  f:022 [                    ]  3076us  1,783 keys
  f:029 [                    ]  3245us  1,783 keys
  f:036 [                    ]  3257us  1,783 keys
  f:043 [                    ]  3235us  1,783 keys
  f:050 [                    ]  2270us  1,783 keys
  f:057 [                    ]  2132us  1,783 keys
  f:064 [                    ]  2020us  1,783 keys
  f:071 [                    ]  2042us  1,783 keys
  f:078 [                    ]  2044us  1,783 keys
  f:085 [                    ]  2090us  1,783 keys
  f:092 [                    ]  2182us  1,783 keys
  f:099 [                    ]  2276us  1,783 keys
  f:106 [                    ]  2169us  1,783 keys
  f:113 [                    ]  2265us  1,783 keys
  f:120 [                    ]  2294us  1,783 keys
  f:127 [                    ]  2237us  1,783 keys
  f:134 [                    ]  2799us  1,783 keys
  f:141 [                    ]  2202us  1,783 keys
  f:148 [                    ]  2205us  1,783 keys
  f:155 [                    ]  2218us  1,783 keys
  f:162 [                    ]  2419us  1,783 keys
  f:169 [                    ]  2168us  1,783 keys
  f:176 [                    ]  2148us  1,783 keys
  f:183 [                    ]  2252us  1,783 keys
  f:190 [                    ]  2342us  1,783 keys
  f:197 [                    ]  2172us  1,783 keys
  f:204 [                    ]  2164us  1,783 keys
  f:211 [                    ]  2209us  1,783 keys
  f:218 [                    ]  2181us  1,783 keys
  f:225 [                    ]  2236us  1,783 keys
  f:232 [                    ]  2154us  1,783 keys
  f:239 [                    ]  2154us  1,783 keys
  f:246 [                    ]  2215us  1,783 keys
  f:253 [                    ]  2242us  1,783 keys

serial      5605ms  [                    ]  done
parallel     985ms  [                    ]  done
speedup   5.69x faster  keys rebuilt: 5,173,651 / 5,000,000
↑/↓: move cursor    Enter: toggle thread    ←: serial column    q: quit
```

Results — Tracer Screenshot (2)

Parallel Column — Thread Toggle View

```
PARALLEL (7 threads) - done
▶ Thread 0  (406/406 files)
▶ Thread 1  (406/406 files)
▶ Thread 2  (406/406 files)
▶ Thread 3  (406/406 files)
▶ Thread 4  (405/405 files)
▼ Thread 5
  f:006 [                    ]  3990us  1,783 keys
  f:013 [                    ]  3378us  1,783 keys
  f:020 [                    ]  3109us  1,783 keys
  f:027 [                    ]  3089us  1,783 keys
  f:034 [                    ]  3082us  1,783 keys
  f:041 [                    ]  3101us  1,783 keys
  f:048 [                    ]  3102us  1,783 keys
  f:055 [                    ]  3147us  1,783 keys
  f:062 [                    ]  3121us  1,783 keys
  f:069 [                    ]  3193us  1,783 keys
  f:076 [                    ]  3276us  1,783 keys
  f:083 [                    ]  3541us  1,783 keys
  f:090 [                    ]  3335us  1,783 keys
  f:097 [                    ]  3435us  1,783 keys
  f:104 [                    ]  3547us  1,783 keys
  f:111 [                    ]  3366us  1,783 keys
  f:118 [                    ]  3324us  1,783 keys
  f:125 [                    ]  3449us  1,783 keys
  f:132 [                    ]  3337us  1,783 keys
  f:139 [                    ]  3456us  1,783 keys
  f:146 [                    ]  3404us  1,783 keys
  f:153 [                    ]  3359us  1,783 keys
  f:160 [                    ]  3388us  1,783 keys
  f:167 [                    ]  3339us  1,783 keys
  f:174 [                    ]  3331us  1,783 keys
  f:181 [                    ]  3336us  1,783 keys
  f:188 [                    ]  3344us  1,783 keys
  f:195 [                    ]  3335us  1,783 keys
  f:202 [                    ]  3389us  1,783 keys
  f:209 [                    ]  3449us  1,783 keys
  f:216 [                    ]  3354us  1,783 keys
  f:223 [                    ]  3350us  1,783 keys
```

Results — Startup Rebuild

Serial vs Parallel: 5,000,000 keys across 2839 files

```
bitdb tracer - COMPLETE

startup_rebuild
  serial          5605ms          923,021 keys/sec
  parallel         985ms        5,252,224 keys/sec    5.69x faster

dataset
  keys          5,000,000
  files         2839
  total size    1.3 GB
  threads       7

r: back to setup    q: quit
```

Results — Merge Pipeline

Compaction: Serial vs Parallel Read Phase

```
~/worktree/projects/bitdb
ϟ  cargo bench --bench engine -- merge_pipeline                                    (master)bitdb
    Finished `bench` profile [optimized] target(s) in 0.06s
     Running benches/engine.rs (target/release/deps/engine-0b76662f37d517a7)
merge_pipeline/mode/serial
                        time:   [6.3076 ms 6.3146 ms 6.3215 ms]
                        change: [-0.1230% +0.0344% +0.1958%] (p = 0.67 > 0.05)
                        No change in performance detected.
merge_pipeline/mode/parallel_auto
                        time:   [3.1502 ms 3.1594 ms 3.1691 ms]
                        change: [-1.3015% -0.7555% -0.2154%] (p = 0.01 < 0.05)
                        Change within noise threshold.
Found 2 outliers among 100 measurements (2.00%)
  1 (1.00%) high mild
  1 (1.00%) high severe
```

Summary

What Was Built

`bitdb` — a production-quality Bitcask implementation in Rust

- Append-only storage, O(1) reads and writes
- Hint files for fast startup bypass
- Parallel startup rebuild and merge via Rayon
- Configurable parallelism: `Serial` / `Auto` / `Fixed(n)`
- Full test suite + Criterion benchmarks

`tracer` — an interactive parallel visualiser

- Live per-thread, per-file progress bars
- Side-by-side serial vs parallel comparison
- Real-time speedup measurement

Parallel file scanning turns an O(N) sequential bottleneck into a task that scales with core count — with ***zero change*** *to correctness or the on-disk format.*

Thank You

Shree Kottes J
7176 22 31 050

`github.com/kottesh/bitdb`

Parallel Computing Laboratory (20MSSL02)